Forensics Git 2

⚠️ **Security Warning**: Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

📋 Challenge Description

The agents interrupted the perpetrator's disk deletion routine. Can you recover this git repo?

💡 Hints

- We think the deletion was interrupted before any git objects were touched

---

🧰 Tools Used

- `7z` – Extract compressed files
- `file` – Determine file type
- `strings` – Extract printable character sequences
- `exiftool` – Read metadata
- `fdisk` – View partition tables
- `dd` – Extract specific partitions
- `mount` – Mount a filesystem as a loopback device
- `tree` – Display directory structure
- `git` – Examine repository history and commits
- `grep` – Filter output
- `cat` – Read file contents

🔍 Complete Solution

Step 1: Download and Extract

Download the disk image named `disk.img.gz` from the challenge page.

Extract the compressed disk image:

```
7z x disk.img.gz
```

This decompresses the gzipped file and creates the raw disk image `disk.img`.

Step 2: Basic File Inspection

First, verify the file type and understand its structure:

```
file disk.img
```

Expected output:

```
disk.img: DOS/MBR boot sector; partition 1 : ID=0x83, active, start-CHS (0x2,0,33), end-CHS (0x263,8,56), startsector 2048, 614400 sectors; partition 2 : ID=0x82, start-CHS (0x263,8,57), end-CHS (0x3ff,15,63), startsector 616448, 524288 sectors; partition 3 : ID=0x83, start-CHS (0x3ff,15,63), end-CHS (0x3ff,15,63), startsector 1140736, 956416 sectors
```

The output reveals a **DOS/MBR boot sector** with **three partitions**:

- Partition 1: Linux (ID=0x83) – Boot partition
- Partition 2: Linux swap (ID=0x82) – Swap space
- Partition 3: Linux (ID=0x83) – Data partition (most likely where user files live)

Step 3: Quick String Search

Before diving deep, let's see if the flag is hiding in plain sight:

```
strings disk.img | grep -i "picoCTF"
```

Output: (no results)

The flag isn't directly visible. Time to roll up our sleeves and dig into the filesystem.

Step 4: Inspect Metadata

Examine the disk image metadata for any hidden clues:

```
exiftool disk.img
```

Expected output:

```
ExifTool Version Number         : 13.50
File Name                       : disk.img
Directory                       : .
File Size                       : 1074 MB
File Modification Date/Time     : 2025:11:19 12:57:00+02:00
File Access Date/Time           : 2026:05:18 21:12:17+02:00
File Inode Change Date/Time     : 2026:05:18 21:12:11+02:00
File Permissions                : -rw-r--r--
Error                           : Unknown file type
```

Nothing suspicious here – the metadata is clean.

Step 5: Examine the Partition Table

Use `fdisk` to get a detailed view of the partition layout:

```
fdisk -l disk.img
```

Expected output:

```
Disk disk.img: 1 GiB, 1073741824 bytes, 2097152 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x610b63c2

Device     Boot   Start     End Sectors  Size Id Type
disk.img1  *       2048  616447  614400  300M 83 Linux
disk.img2        616448 1140735  524288  256M 82 Linux swap / Solaris
disk.img3       1140736 2097151  956416  467M 83 Linux
```

The third partition (`disk.img3`) is the largest – this is where user data is likely stored. The first two are boot and swap partitions, which we can ignore.

Step 6: Extract the Linux Partition

We'll extract the third partition using `dd` . The parameters are:

- `skip=1140736` : number of sectors to skip (start of partition 3)
- `count=956416` : number of sectors to copy (size of partition 3)

```
dd if=disk.img of=linux_partition.dd bs=512 skip=1140736 count=956416 status=progress
```

Output:

```
956416+0 records in
956416+0 records out
489684992 bytes (490 MB, 467 MiB) copied, X.XXXXX s, XX.X MB/s
```

Step 7: Verify the Extracted Partition

Check the type of the extracted partition:

```
file linux_partition.dd
```

Output:

```
linux_partition.dd: Linux rev 1.0 ext4 filesystem data, UUID=7a00e9da-98f8-4f0f-b257-
95edf422d902 (needs journal recovery) (extents) (64bit) (large files) (huge files)
```

Perfect! It's a valid **ext4 filesystem**. The "needs journal recovery" message is normal for a filesystem that wasn't properly unmounted.

Step 8: Mount the Extracted Partition

Create a mount point and mount the partition as a loopback device:

```
sudo mkdir /mnt/git2
sudo mount -o loop linux_partition.dd /mnt/git2
```

> **Note**: The `-o loop` option tells Linux to treat the file as a block device.

Step 9: Explore the Filesystem

List the contents of the root directory:

```
ls -alh /mnt/git2
```

Output:

```
total 45K
drwxr-xr-x 22 root root 1.0K Nov 19 10:38 .
drwxr-xr-x 13 root root 4.0K May 14 17:22 ..
drwxr-xr-x  2 root root 4.0K Nov 19 10:39 bin
drwxr-xr-x  2 root root 1.0K Nov 11  2025 boot
drwxr-xr-x  2 root root 1.0K Nov 11  2025 dev
drwxr-xr-x 30 root root 3.0K Nov 19 10:39 etc
drwxr-xr-x  3 root root 1.0K Nov 11  2025 home
...
```

Now let's navigate to the `home` directory – this is where user data is typically stored:

```
tree /mnt/git2/home -alh
```

Output:

```
[1.0K]  /mnt/git2/home
└── [1.0K]  ctf-player
    └── [1.0K]  Code
        └── [1.0K]  killer-chat-app
            ├── [1.0K]  .git
            │   ├── [  20]  COMMIT_EDITMSG
            │   ├── [  23]  HEAD
            │   ├── [1.0K]  branches
            │   ├── [ 150]  config
            │   ├── [  73]  description
            │   ├── [1.0K]  hooks
            │   │   ├── [ 478]  applypatch-msg.sample
```

```
            │   │   ├── [ 896]  commit-msg.sample
...
```

> **Key finding**: A `.git` repository exists at `/home/ctf-player/Code/killer-chat-app/.git`! This is where the deleted secrets might be hiding.

Step 10: Investigate the Git Repository

Navigate to the repository:

```
cd /mnt/git2/home/ctf-player/Code/killer-chat-app/.git
```

First, let's see if there are any commits:

```
git log --oneline
```

Output:

```
fatal: your current branch 'master' does not have any commits yet
```

Interesting! The branch is empty – but wait, the hint said *"the deletion was interrupted before any git objects were touched"*. That means the objects might still exist, even if the branch pointer is gone.

Step 11: Check the Git Logs (Reflog)

Git keeps a log of all reference changes in the `logs` directory. Let's explore it:

```
tree logs
```

Output:

```
logs
├── HEAD
└── refs
    └── heads
        └── master
```

Bingo! The `logs/refs/heads/master` file contains the commit history. Let's read it:

```
cat logs/refs/heads/master
```

Output:

```
0000000000000000000000000000000000000000 2c0a9b2b15dce92f800393d5030c7454efc278ae ctf-player <ctf-player@example.com> 1763549240 +0000  commit (initial): Add netcat scripts
2c0a9b2b15dce92f800393d5030c7454efc278ae 26b809e0c41d8421f1126ed3a4eb06ad66e6d90a ctf-player <ctf-player@example.com> 1763549240 +0000  commit: Add video game chat log
26b809e0c41d8421f1126ed3a4eb06ad66e6d90a 5827632e046a80a1e0d7b4fc5c7800dd539baeaf ctf-player <ctf-player@example.com> 1763549240 +0000  commit: Add TV show chat log
5827632e046a80a1e0d7b4fc5c7800dd539baeaf e80b38b3322a5ba32ac07076ef5eeb4a59449875 ctf-player <ctf-player@example.com> 1763549240 +0000  commit: Add secret hideout chat log
e80b38b3322a5ba32ac07076ef5eeb4a59449875 2151ef0ccc15aed1ab88e1afdc7484aaeff211c4 ctf-player <ctf-player@example.com> 1763549240 +0000  commit: Remove secret hideout log
2151ef0ccc15aed1ab88e1afdc7484aaeff211c4 01533f718556a0e59f1467dae4fa462eed82c2a1 ctf-player <ctf-player@example.com> 1763549240 +0000  commit: Add random chat log
```

This is a goldmine! We can see the entire commit history, including the **"Remove secret hideout log"** commit. That's exactly what we're looking for!

Step 12: Extract Deleted Content

Let's examine the commit before the deletion (the one that added the secret hideout log):

```
git show e80b38b3322a5ba32ac07076ef5eeb4a59449875 --pretty=''
```

Output:

```
diff --git a/logs/3.txt b/logs/3.txt
new file mode 100644
index 0000000..7178644
--- /dev/null
+++ b/logs/3.txt
@@ -0,0 +1,3 @@
+Rex: Meet at the old arcade basement for the secret hideout.
+Jay: Ask Rusty at the door and use password picoCTF{<REDACTED>}.
+Rex: Bring the decoder map so we can plan the route.
```

Now let's look at the deletion commit:

```
git show 2151ef0ccc15aed1ab88e1afdc7484aaeff211c4 --pretty=''
```

Output:

```
diff --git a/logs/3.txt b/logs/3.txt
deleted file mode 100644
index 7178644..0000000
--- a/logs/3.txt
+++ /dev/null
@@ -1,3 +0,0 @@
-Rex: Meet at the old arcade basement for the secret hideout.
-Jay: Ask Rusty at the door and use password picoCTF{<REDACTED>}.
-Rex: Bring the decoder map so we can plan the route.
```

> **Key finding**: The flag appears in the deleted file `logs/3.txt` ! The password is `picoCTF{<REDACTED>}` .

The perpetrator tried to delete the evidence, but the disk deletion was interrupted. Git's object model preserved everything, and the reflog allowed us to recover the deleted commit.

🔗 Alternative Git Commands for Forensic Analysis

If the branch pointer is missing, here are alternative ways to recover lost commits:

Command	Purpose
`git fsck --lost-found`	Find dangling commits (not referenced by any branch or tag)
`git reflog`	Show reference log (where HEAD pointed over time)
`git log --all`	Show all commits, including those not in current branch
`git show <hash>`	Display a specific commit's changes
`git grep "pattern" $(git rev-list --all)`	Search all commits for a pattern

Command	Purpose
`git checkout <hash>`	Temporarily switch to a lost commit
`git cherry-pick <hash>`	Apply a lost commit to current branch

✅ Final Result

```
picoCTF{<REDACTED>}
```

> **Note**: The actual flag is redacted here. In the real challenge, the `git show` commands will reveal the complete flag.

🧠 Key Concepts

Concept	Description
Disk Image (DD)	A raw sector-by-sector copy of a storage device.
MBR Partition Table	A structure that divides a disk into multiple partitions (max 4 primary).
Partition Extraction with `dd`	Using `skip` and `count` to isolate a specific partition from a disk image.
ext4 Filesystem	The native Linux filesystem, commonly used for system and data partitions.
Loopback Mount	Mounting a file as a block device using `mount -o loop`.
Git Reflog	A log of where HEAD and branch references have pointed over time.
Dangling Commits	Commits that are no longer referenced by any branch or tag but still exist in the object database.

Concept	Description
Recovering Deleted Files	Using `git show <commit>` to retrieve content that was deleted in a later commit.
Forensic Git Analysis	Examining `.git` directories to recover deleted files, commit history, and hidden information.

💡 Pro Tips

1. **Always check the Git reflog** – Even if a branch is deleted or reset, the reflog (`logs/refs/heads/*`) often preserves the history.
2. **Git objects are rarely deleted immediately** – The hint explicitly stated the deletion was interrupted, meaning objects are still intact.
3. **The `logs` directory is your best friend** – It contains a complete history of all reference changes, even if branches are gone.
4. **Use `git show <hash>` to inspect commits** – This works even if the commit is no longer in the current branch.
5. **Look for deletion commits** – Commits with messages like “Remove”, “Delete”, or “Cleanup” often contain the removed content in their diff.
6. **When dealing with disk images**, always check for `.git` , `.svn` , `.hg` , or other version control artifacts — they are goldmines for forensic analysis.